



廣東工業大學

实验报告

课程名称 EDA 技术和工具

实验名称 基于 Bit-Fusion 的 PE 单元设计

学生学院 集成电路学院

专业班级 2023 级集成 3 班

学 号 3123009486 3123009487 3123009499

学生姓名 侯艾辰 黄海鸿 宋嘉威

指导老师 胡湘宏

2025 年 6 月 28 日

目录

第 1 部分 题目说明.....	1
1.1 题目介绍.....	1
1.2 设计目标.....	1
1.3 成员分工.....	1
第 2 部分 前端设计.....	2
2.1 基本乘法单元 BitBrick 设计	2
2.2 融合 PE 架构设计.....	3
第 3 部分 实验仿真结果.....	6
3.1 Int2 无符号输入.....	6
3.2 Int2 有符号输入.....	7
3.3 Int4 无符号输入.....	8
3.4 Int4 有符号输入.....	9
3.5 Int8 无符号输入.....	10
3.6 Int8 有符号输入.....	11
第 4 部分 后端综合与版图设计	12
4.1 逻辑综合.....	12
4.1.1 逻辑综合原理.....	12
4.1.2 综合脚本.....	12
4.1.3 时序约束脚本.....	13

4.1.4 综合结果报告	13
4.1.5 综合原理图	15
4.2 版图布局规划	16
4.2.1 设置芯片形状以及 corner, pad 位置	16
4.2.2 Filler 填充以及 pg connect	18
4.2.3 Congestion check	18
4.2.4 PNS	19
4.2.5 Power rail commit	19
4.2.6 IR drop 分析	20
4.3 布局	21
4.3.1 标准单元布局	21
4.3.2 布局结果报告	22
4.4 时钟树综合	24
4.4.1 CTS	24
4.4.2 时序报告	24
4.5 布线	25
4.5.1 布线实现	25
4.5.2 时序分析	26
4.6 DRC 验证	26
4.7 LVS 验证	26
4.8 版图文件导出	27

第 5 部分 实验过程中遇到的问题	28
第 6 部分 总结.....	30
第 7 部分 引用文献.....	30

第1部分 题目说明

1.1 题目介绍

选其中一种或多种：bit fusion (INT8/4/2)、bit-serial (INT1-INT8)，设计乘法 PE 单元支持 INT8/4/2 精度的乘法计算。

1.2 设计目标

本课题设计在参考论文[1]所提出的 Bit_Fusion 理论,利用若干个基础 2bit 乘法单元 BitBrick 融合成一个 Fusion_PE 以支持计算多位的（无）有符号数乘法计算。

1.3 成员分工

黄海鸿：负责后端综合以及版图设计。

宋嘉威：负责算法基础设计。

侯艾辰：负责程序优化与仿真验证。

第2部分 前端设计

2.1 基本乘法单元 BitBrick 设计

```
1 module BitBrick(  
2     input clk,                // 新增时钟信号  
3     input [1:0] a,  
4     input [1:0] b,  
5     input s_a,  
6     input s_b,  
7     output [3:0] product // 改为寄存器输出  
8 );  
9  
10 reg [2:0] a_ext, b_ext;  
11 reg [5:0] temp_product;  
12  
13 always @(posedge clk) begin  
14     // 符号扩展  
15     a_ext[2] <= s_a & a[1];  
16     a_ext[1:0] <= a[1:0];  
17  
18     b_ext[2] <= s_b & b[1];  
19     b_ext[1:0] <= b[1:0];  
20  
21     // 有符号乘法  
22     temp_product <= $signed(a_ext) * $signed(b_ext);  
23  
24     // 输出结果  
25  
26 end  
27 assign product = temp_product[3:0];  
28 endmodule
```

定义基本乘法单元 BitBrick，其输入接口 a,b 为输入的两个 2bit 数；输入端口 s_a, s_b 分别判定 a,b 是否为符号数（符号数范围为-2~1，无符号数范围为 0~3），其中 s=0 代表无符号数，s=1 代表有符号数；输出端口 product 为 4bit 的两数乘积结果。

该计算单元通过将乘数 a,b 扩展至 3bit（扩展结果分别为 a_ext 与 b_ext，对两者进行有符号乘法得到 6bit 乘积中间值 temp_product，取低四位得到输出乘积 product。

2.2 融合 PE 架构设计

```
3 module F_PE(  
4     input clk,  
5     input [7:0] a,  
6     input [7:0] b,  
7     input s_a,  
8     input s_b,  
9     input [1:0] bitwidth,    // 00:2-bit, 01:4-bit, 10/11:8-bit  
0     output [15:0] result  
1 );  
2  
3 wire [3:0] bb_product [15:0];  
4 reg [7:0] product_4bit [3:0];  
5 reg [15:0] product_8bit [15:0];  
6  
7 reg s_a0,s_a4,s_a5,s_a12,s_a13,s_a14,s_a15;  
8 reg s_b0,s_b1,s_b3,s_b5,s_b7,s_b11,s_b15;  
9  
0 reg [3:0] result_2bit;  
1 reg [7:0] result_4bit;  
2 reg [15:0] result_8bit;  
3  
4  
5 wire [7:0] res_4bit1    = product_4bit[0] + product_4bit[1];  
6 wire [7:0] res_4bit2    = product_4bit[2] + product_4bit[3];  
7  
8 wire [15:0] res_8bit1_1  = product_8bit[0]  + product_8bit[1];  
9 wire [15:0] res_8bit2_1  = product_8bit[2]  + product_8bit[3];  
0 wire [15:0] res_8bit3_1  = product_8bit[4]  + product_8bit[5];  
1 wire [15:0] res_8bit4_1  = product_8bit[6]  + product_8bit[7];  
2 wire [15:0] res_8bit5_1  = product_8bit[8]  + product_8bit[9];  
3 wire [15:0] res_8bit6_1  = product_8bit[10] + product_8bit[11];  
4
```

融合 PE 单元 F_PE 输入端口 a,b 为最多支持 8bit 输入的乘数（未达到 8bit 在高位补零）；输入端口 s_a, s_b 分别判定乘数 a,b 是否为有符号数（定义 s=0 为无符号数，s=1 为有符号数）；输入端口 bitwidth 用于决定此时 F_PE 处理多少位的乘法计算（00 : 2bit, 01 : 4bit, 10/11 : 8bit）；输出端口 result 为最终的乘积结果。

```

2 wire [15:0] res_8bit5_1 = product_8bit[8] + product_8bit[9];
3 wire [15:0] res_8bit6_1 = product_8bit[10] + product_8bit[11];
4 wire [15:0] res_8bit7_1 = product_8bit[12] + product_8bit[13];
5 wire [15:0] res_8bit8_1 = product_8bit[14] + product_8bit[15];
6
7 wire [15:0] res_8bit1_2 = res_8bit1_1 + res_8bit2_1;
8 wire [15:0] res_8bit2_2 = res_8bit3_1 + res_8bit4_1;
9 wire [15:0] res_8bit3_2 = res_8bit5_1 + res_8bit6_1;
0 wire [15:0] res_8bit4_2 = res_8bit7_1 + res_8bit8_1;
1
2 wire [15:0] res_8bit1_3 = res_8bit1_2 + res_8bit2_2;
3 wire [15:0] res_8bit2_3 = res_8bit3_2 + res_8bit4_2;
4
5 BitBrick bb0(.clk(clk), .a(a[1:0]), .b(b[1:0]), .s_a(s_a0), .s_b(s_b0), .p
6 BitBrick bb1(.clk(clk), .a(a[1:0]), .b(b[3:2]), .s_a(1'b0), .s_b(s_b1), .produc
7 BitBrick bb2(.clk(clk), .a(a[1:0]), .b(b[5:4]), .s_a(1'b0), .s_b(1'b0), .produc
8 BitBrick bb3(.clk(clk), .a(a[1:0]), .b(b[7:6]), .s_a(1'b0), .s_b(s_b3), .produc
9 BitBrick bb4(.clk(clk), .a(a[3:2]), .b(b[1:0]), .s_a(s_a4), .s_b(1'b0), .produc
0 BitBrick bb5(.clk(clk), .a(a[3:2]), .b(b[3:2]), .s_a(s_a5), .s_b(s_b5), .produc
1 BitBrick bb6(.clk(clk), .a(a[3:2]), .b(b[5:4]), .s_a(1'b0), .s_b(1'b0), .produc
2
35 BitBrick bb10(.clk(clk), .a(a[5:4]), .b(b[5:4]), .s_a(1'b0), .s_b(1'b
36 BitBrick bb11(.clk(clk), .a(a[5:4]), .b(b[7:6]), .s_a(1'b0), .s_b(s_b
37 BitBrick bb12(.clk(clk), .a(a[7:6]), .b(b[1:0]), .s_a(s_a12), .s_b(1'
38 BitBrick bb13(.clk(clk), .a(a[7:6]), .b(b[3:2]), .s_a(s_a13), .s_b(1'
39 BitBrick bb14(.clk(clk), .a(a[7:6]), .b(b[5:4]), .s_a(s_a14), .s_b(1'
40 BitBrick bb15(.clk(clk), .a(a[7:6]), .b(b[7:6]), .s_a(s_a15), .s_b(s_
41
42 always @(posedge clk) begin
43     s_a0 <= (bitwidth == 2'b00) && s_a;
44     s_a4 <= (bitwidth == 2'b01) && s_a;
45     s_a5 <= (bitwidth == 2'b01) && s_a;
46     s_a12 <= (bitwidth[1]) && s_a;
47     s_a13 <= (bitwidth[1]) && s_a;
48     s_a14 <= (bitwidth[1]) && s_a;
49     s_a15 <= (bitwidth[1]) && s_a;
50
51
52     s_b0 <= (bitwidth == 2'b00) && s_b;
53     s_b1 <= (bitwidth == 2'b01) && s_b;
54     s_b3 <= (bitwidth[1]) && s_b;
55     s_b5 <= (bitwidth == 2'b01) && s_b;
56     s_b7 <= (bitwidth[1]) && s_b;
57     s_b11 <= (bitwidth[1]) && s_b;
58     s_b15 <= (bitwidth[1]) && s_b;
59
60     product_4bit[0] <= {4'b0000, bb_product[0]};
61     product_4bit[1] <= (s_b1 && b[3] && (bb_product[1] != 0)) ? {2'b
62     product_4bit[2] <= (s_a4 && a[3] && (bb_product[4] != 0)) ? {2'b
63     product_4bit[3] <= {bb_product[5], 4'b0000};
64
65     product_8bit[0] <= {12'h0, bb_product[0]};
66     product_8bit[1] <= {10'h0, bb_product[1], 2'h0};
67     product_8bit[2] <= {8'h0, bb_product[2], 4'h0};
68     product_8bit[3] <= (s_b3 && b[7] && (bb_product[3] != 0)) ? {6'h

```



```

product_8bit[2] <= {8'h0,bb_product[2],4'h0};
product_8bit[3] <= (s_b3 && b[7] && (bb_product[3] != 0)) ? {6'h3f,bb_product[3],4'h0};
product_8bit[4] <= {10'h0,bb_product[4],2'h0};
product_8bit[5] <= {8'h0,bb_product[5],4'h0};
product_8bit[6] <= {6'h0,bb_product[6],6'h0};
product_8bit[7] <= (s_b7 && b[7] && (bb_product[7] != 0)) ? {4'hf,bb_product[7],4'h0};
product_8bit[8] <= {8'h0,bb_product[8],4'h0};
product_8bit[9] <= {6'h0,bb_product[9],6'h0};
product_8bit[10] <= {4'h0,bb_product[10],8'h0};
product_8bit[11] <= (s_b11 && b[7] && (bb_product[11] != 0)) ? {2'h3,bb_product[11],4'h0};
product_8bit[12] <= (s_a12 && a[7] && (bb_product[12] != 0)) ? {6'h3f,bb_product[12],4'h0};
product_8bit[13] <= (s_a13 && a[7] && (bb_product[13] != 0)) ? {4'hf,bb_product[13],4'h0};
product_8bit[14] <= (s_a14 && a[7] && (bb_product[14] != 0)) ? {2'h3,bb_product[14],4'h0};
product_8bit[15] <= {bb_product[15],12'h0};

result_2bit <= bb_product[0];
result_4bit <= res_4bit1 + res_4bit2;
result_8bit <= res_8bit1_3 + res_8bit2_3;

end
assign result = bitwidth[1] ? result_8bit :
               bitwidth[0] ? {8'b0, result_4bit} : {12'b0, result_2bit};
endmodule

```

该架构采用论文当中的 Bit-fusion 方法，将多位乘数乘法分解为 2bit 乘数乘法之和之间的移位和，下面是该方法的详细介绍：

1. 将 8bit 的操作数分解为 4 个 2bit 部分，从高位到低位依次编号 3~0,记为 a[i]与 b[j]
2. 采用 16 个 BitBrick 编号为 bb0~bb15，其中 bb[4i+j]处理 a[i]与 b[j]之间的乘法
3. 由于将多位有符号数分解为 2 位有符号数计算只有最高 2 位可以作为有符号数计算，因此定义 s_a0,s_a4,...,s_a15, s_b0,s_b1,...,s_b15 来控制对应 BitBrick 是否采用符号计算。
4. 定义 product_4bit 与 product_8bit 将对应 bb 的乘积结果进行移位或补 0（补码可能补 1）以将数据分别规范到 8bit 与 16bit。
5. 计算 product 之和得到 result，进行结果输出。

第3部分 实验仿真结果

3.1 Int2 无符号输入

```
repeat(5)begin
#5000

    a_in = {$random}%4;
    b_in = {$random}%4;
    s_a = 1'b0;
    s_b = 1'b0;
    bitwidth = 2'b00;
end
#5000
a_in = 0;
b_in = 0;
end
```

> a_in[7:0]	00	XX	00	01				
> b_in[7:0]	01	XX	01	03	01	02	01	
🔧 s_a	0							
🔧 s_b	0							
> bitwidth[1:0]	0	X						
> result_out[15:0]	000X	XXXX	0000	0003	0001	0002	0001	
🔧 clk	0							

3.2 Int2 有符号输入

```
initial begin
#100

repeat(5) begin
#5000

    a_in = {$random}%4;
    b_in = {$random}%4;
    s_a = 1'b1;
    s_b = 1'b1;
    bitwidth = 2'b00;
end
#5000
a_in = 0;
b_in = 0;
end
```

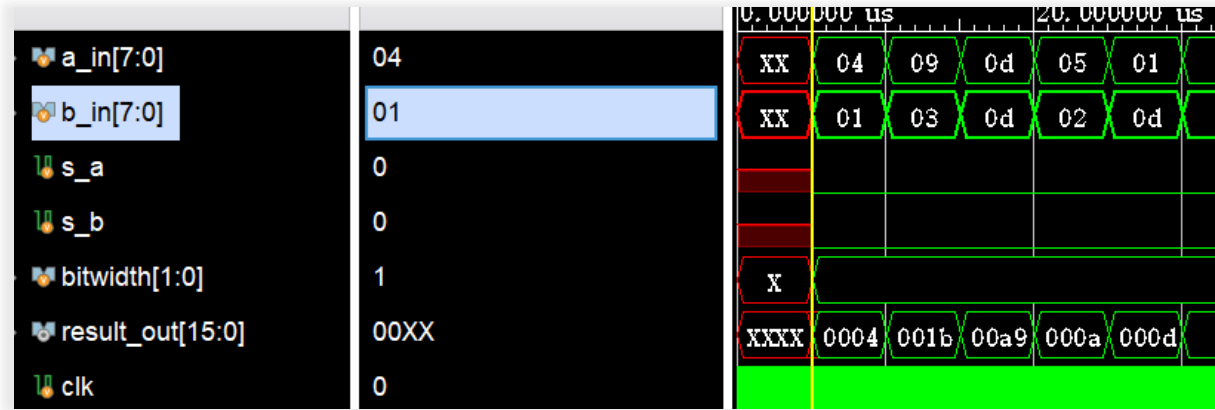


3.3 Int4 无符号输入

```
initial begin
#100

repeat(5)begin
#5000

    a_in = {$random}%16;
    b_in = {$random}%16;
    s_a = 1'b0;
    s_b = 1'b0;
    bitwidth = 2'b01;
end
#5000
a_in = 0;
b_in = 0;
end
```



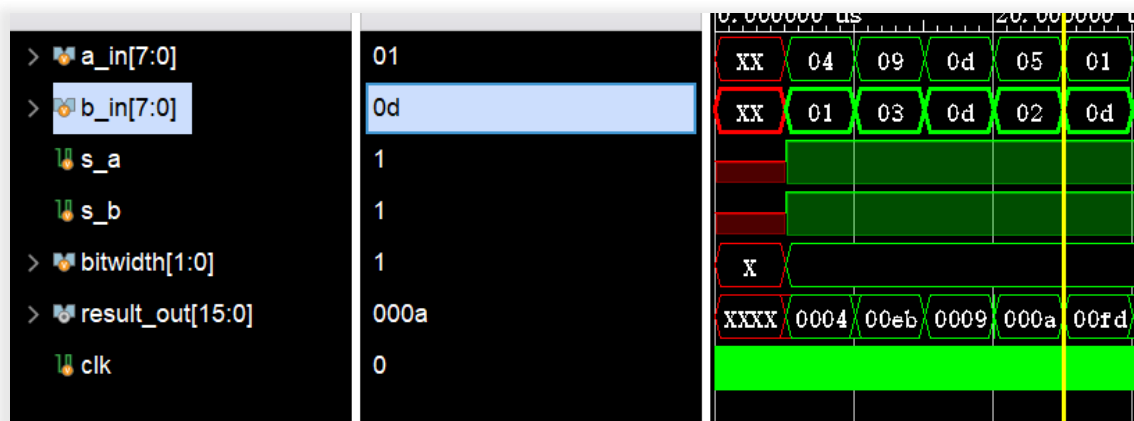
3.4 Int4 有符号输入

```
initial begin
#100

repeat(5)begin
#5000

    a_in = {$random}%16;
    b_in = {$random}%16;
    s_a = 1'b1;
    s_b = 1'b1;
    bitwidth = 2'b01;
    end
#5000
    a_in = 0;
    b_in = 0;

end
```

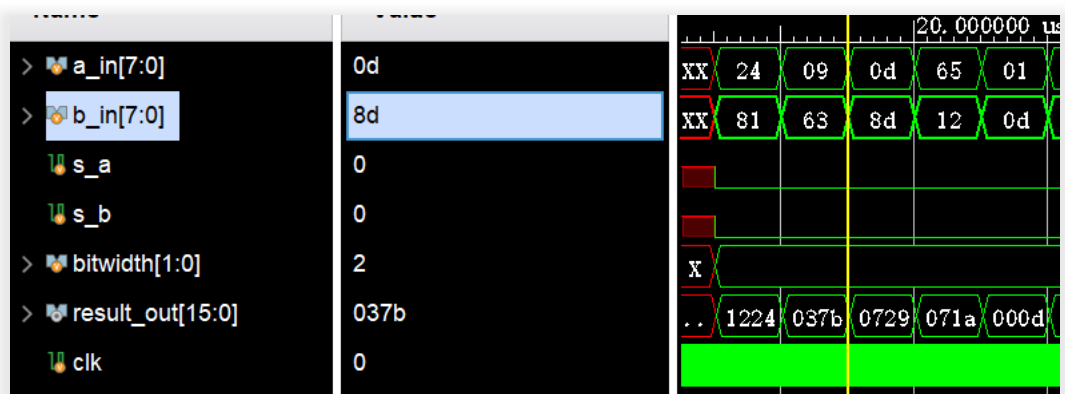


3.5 Int8 无符号输入

```
initial begin
    #100

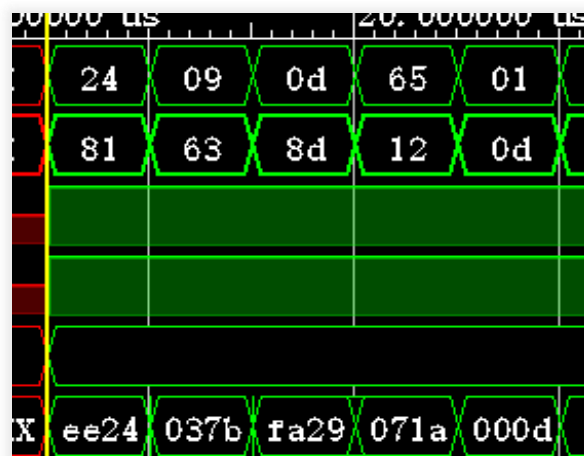
    repeat(5)begin
        #5000

        a_in = {$random}%256;
        b_in = {$random}%256;
        s_a = 1'b0;
        s_b = 1'b0;
        bitwidth = 2'b10;
    end
    #5000
    a_in = 0;
    b_in = 0;
end
```



3.6 Int8 有符号输入

```
initial begin
#100
repeat(5)begin
#5000
    a_in = {$random}%256;
    b_in = {$random}%256;
    s_a = 1'b1;
    s_b = 1'b1;
    bitwidth = 2'b10;
end
#5000
a_in = 0;
b_in = 0;
end
```



第4部分 后端综合与版图设计

4.1 逻辑综合

4.1.1 逻辑综合原理

逻辑综合是数字电路设计中的关键步骤，其原理是将高级硬件描述语言（如 Verilog 或 VHDL）编写的寄存器传输级（RTL）代码，通过算法转换为门级网表，实现功能与工艺的映射。综合过程需要寻求时序与面积、功耗与时序的平衡，并加强电路的测试性能。通过依赖单元库的支持，综合工具能够在满足设计要求的前提下，找到最佳的逻辑网络结构，从而实现对电路功能、速度、面积等方面的综合优化，是连接前端设计与后端物理实现的核心桥梁。

4.1.2 综合脚本

```
read_verilog {F_PE.v BitBrick.v}
current_design F_PE

link
check_design

source F_PE.con
check_timing
source F_PE.pcon

compile_ultra -spg -retime -scan

source report.tcl

list_licenses
report_hierarchy -noleaf
```


4.1.3 时序约束脚本

```
1 # F_PE_constraints.tcl
2
3 # clock
4 create_clock -period 3.8 -name clk [get_ports clk]
5 set_clock_uncertainty -setup 0.1 [get_clocks clk]
5 set_clock_transition -rise -max 0.1 [get_clocks clk]
7 set_clock_transition -fail -max 0.1 [get_clocks clk]
8
9 # input delay
9 set_input_delay 0.1 -clock clk [get_ports {a[*] b[*] s_a s_b bitwidth[*]}]
1
2 # output delay
3 set_output_delay 0.1 -clock clk [get_ports result]
4
5 group_path -name INPUTS -from [all_inputs]
5 group_path -name OUTPUTS -to [all_outputs]
7 group_path -name BITBRICK -through [get_cells bb*]
8
9 write_sdc -version 2.0 -nosplit F_PE_constraints.sdc
10
```

4.1.4 综合结果报告

在运行综合脚本，打印出综合报告如下：

(1) 时序报告（此处只列出其中一条路径）：

Operating Conditions: cb13fs120_tsmc_max Library: cb13fs120_tsmc_max		
Startpoint: bb12/temp_product_reg[0] (rising edge-triggered flip-flop clocked by clk)		
Endpoint: R_176 (rising edge-triggered flip-flop clocked by clk)		
Path Group: BITBRICK		
Path Type: max		
Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
bb12/temp_product_reg[0]/CP (sdnrq1)	0.00	0.00 r
bb12/temp_product_reg[0]/Q (sdnrq1)	0.35 z	0.35 f
bb12/product[0] (BitBrick_3)	0.00 z	0.35 f
U449/ZN (nr04d0)	0.18 z	0.53 r
*cell*5025/Z (or02d1)	0.17 z	0.69 r
U452/S (ad01d0)	0.37 z	1.07 f
U135/S (ad01d0)	0.37 z	1.43 r
R_176/D (sdnrb1)	0.00 z	1.43 r
data arrival time		1.43
clock clk (rise edge)	3.80	3.80
clock network delay (ideal)	0.00	3.80
clock uncertainty	-0.10	3.70
R_176/CP (sdnrb1)	0.00	3.70 r
library setup time	-0.20	3.50
data required time		3.50

data required time		3.50
data arrival time		-1.43

slack (MET)		2.07

(2) 面积报告:

```

*****
Report : area
Design : F_PE
Version: K-2015.06
Date   : Mon Jun 23 17:37:08 2025
*****

Library(s) Used:

    gtech (File: /opt/Synopsys/Synplify2015/libraries/syn/gtech.db)

Number of ports:                2201
Number of nets:                  4525
Number of cells:                 1971
Number of combinational cells:   1248
Number of sequential cells:      639
Number of macros/black boxes:    0
Number of buf/inv:               198
Number of references:            9

Combinational area:              0.000000
Buf/Inv area:                    0.000000
Noncombinational area:           0.000000
Macro/Black Box area:            0.000000
Net Interconnect area:          925.519391

Total cell area:                 0.000000
Total area:                      925.519391

```

(3) 功耗报告:

```

}2
}3 Global Operating Voltage = 1.08
}4 Power-specific unit information :
}5   Voltage Units = 1V
}6   Capacitance Units = 1.000000pf
}7   Time Units = 1ns
}8   Dynamic Power Units = 1mW   (derived from V,C,T units)
}9   Leakage Power Units = 1pW
}10
}11
}12 -----
}13 Hierarchy          Switch  Int    Leak   Total
}14                   Power   Power  Power  Power  %
}15 -----
}16 F_PE               2.23e-02  0.000  0.000  2.23e-02 100.0
}17   C173 (*SELECT_OP_3.16_3.1_16) 3.67e-04 0.000  0.000  3.67e-04 1.6
}18   add_107 (*ADD_UNOS_OP_16_16_16) 8.04e-04 0.000  0.000  8.04e-04 3.6
}19   add_43 (*ADD_UNOS_OP_16_16_16) 5.19e-04 0.000  0.000  5.19e-04 2.3
}20   add_42 (*ADD_UNOS_OP_16_16_16) 5.73e-04 0.000  0.000  5.73e-04 2.6
}21   add_40 (*ADD_UNOS_OP_16_16_16) 2.71e-04 0.000  0.000  2.71e-04 1.2
}22   add_39 (*ADD_UNOS_OP_16_16_16) 2.37e-04 0.000  0.000  2.37e-04 1.1
}23   add_38 (*ADD_UNOS_OP_16_16_16) 2.63e-04 0.000  0.000  2.63e-04 1.2

```

4.1.5 综合原理图

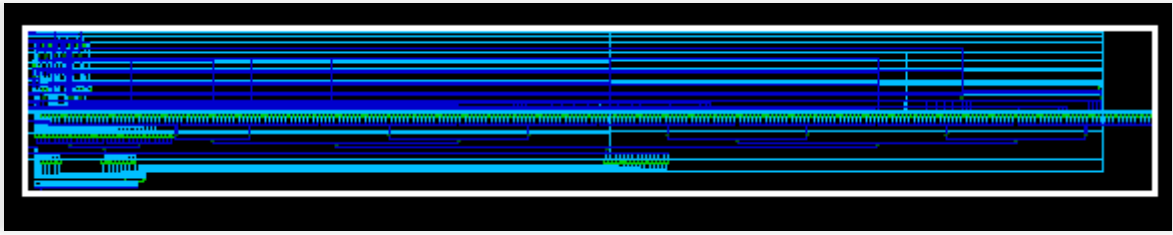


图 1：全局图

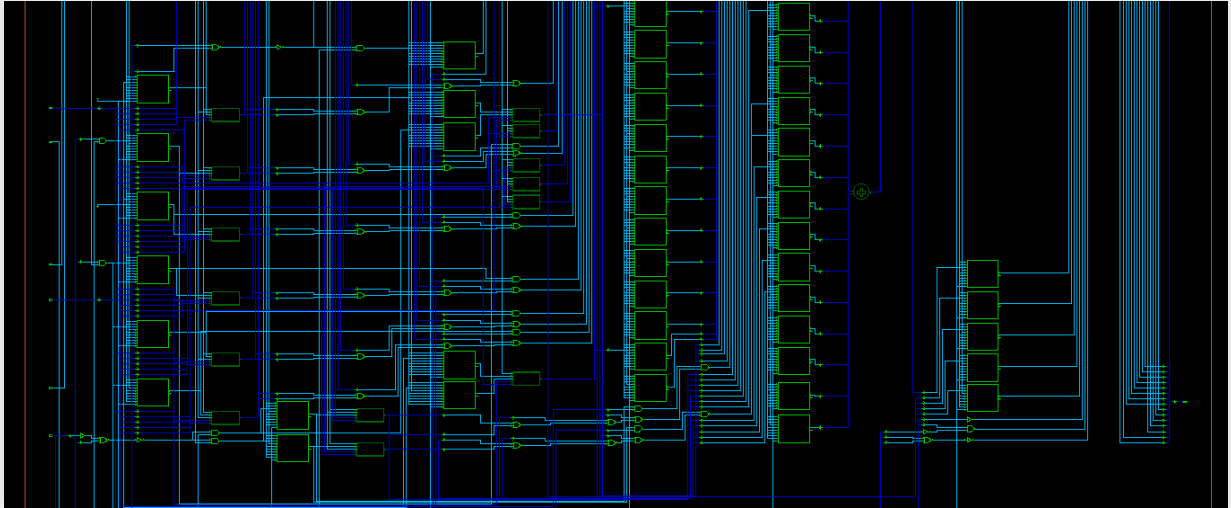
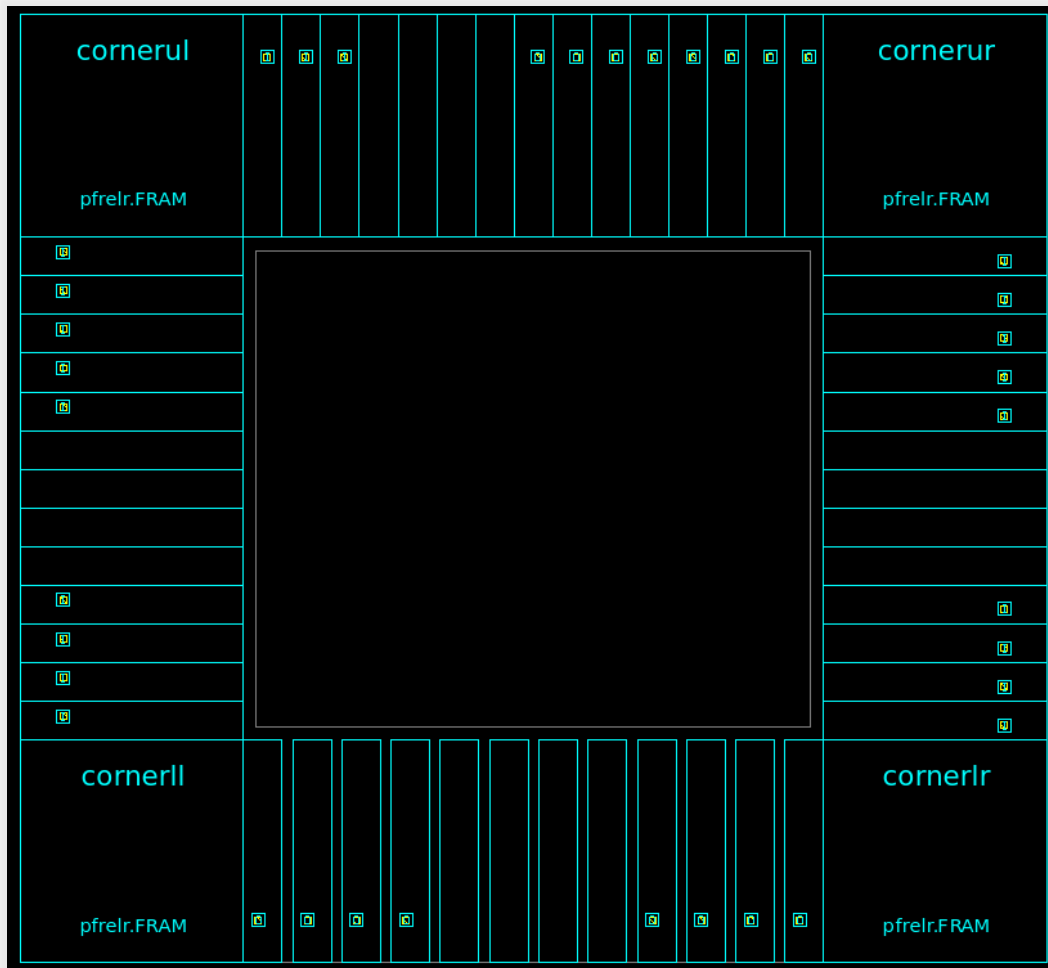


图 2：局部放大图

4.2 版图布局规划

4.2.1 设置芯片形状以及 corner, pad 位置



corner 和 pad 位置如下：

```
# Create corners and P/G pads
create_cell {cornerll cornerlr cornerul cornerur} pfreir
create_cell {vss1left vss1right vss1top vss1bottom} pv0i
create_cell {vdd1left vdd1right vdd1top vdd1bottom} pvdi
create_cell {vss2left vss2right vss2top vss2bottom} pv0a
create_cell {vdd2left vdd2right vdd2top vdd2bottom} pvda

# Define corner pad locations
set_pad_physical_constraints -pad_name "cornerul" -side 1
set_pad_physical_constraints -pad_name "cornerur" -side 2
set_pad_physical_constraints -pad_name "cornerlr" -side 3
set_pad_physical_constraints -pad_name "cornerll" -side 4
```

```

# Left side
set_pad_physical_constraints -pad_name "pad_a_0" -side 1 -order 1
set_pad_physical_constraints -pad_name "pad_a_1" -side 1 -order 2
set_pad_physical_constraints -pad_name "pad_a_2" -side 1 -order 3
set_pad_physical_constraints -pad_name "pad_a_3" -side 1 -order 4
set_pad_physical_constraints -pad_name "vdd2left" -side 1 -order 5
set_pad_physical_constraints -pad_name "vdd1left" -side 1 -order 6
set_pad_physical_constraints -pad_name "vss1left" -side 1 -order 7
set_pad_physical_constraints -pad_name "vss2left" -side 1 -order 8
set_pad_physical_constraints -pad_name "pad_a_4" -side 1 -order 9
set_pad_physical_constraints -pad_name "pad_a_5" -side 1 -order 10
set_pad_physical_constraints -pad_name "pad_a_6" -side 1 -order 11
set_pad_physical_constraints -pad_name "pad_a_7" -side 1 -order 12
set_pad_physical_constraints -pad_name "pad_s_a" -side 1 -order 13

```

```

# Top side
set_pad_physical_constraints -pad_name "pad_bw_0" -side 2 -order 1
set_pad_physical_constraints -pad_name "pad_bw_1" -side 2 -order 2
set_pad_physical_constraints -pad_name "pad_clk" -side 2 -order 3
set_pad_physical_constraints -pad_name "vdd2top" -side 2 -order 4
set_pad_physical_constraints -pad_name "vdd1top" -side 2 -order 5
set_pad_physical_constraints -pad_name "vss1top" -side 2 -order 6
set_pad_physical_constraints -pad_name "vss2top" -side 2 -order 7
set_pad_physical_constraints -pad_name "pad_res_8" -side 2 -order 8
set_pad_physical_constraints -pad_name "pad_res_9" -side 2 -order 9
set_pad_physical_constraints -pad_name "pad_res_10" -side 2 -order 10
set_pad_physical_constraints -pad_name "pad_res_11" -side 2 -order 11
set_pad_physical_constraints -pad_name "pad_res_12" -side 2 -order 12
set_pad_physical_constraints -pad_name "pad_res_13" -side 2 -order 13
set_pad_physical_constraints -pad_name "pad_res_14" -side 2 -order 14
set_pad_physical_constraints -pad_name "pad_res_15" -side 2 -order 15

```

```

# Right side
set_pad_physical_constraints -pad_name "pad_b_0" -side 3 -order 1
set_pad_physical_constraints -pad_name "pad_b_1" -side 3 -order 2
set_pad_physical_constraints -pad_name "pad_b_2" -side 3 -order 3
set_pad_physical_constraints -pad_name "pad_b_3" -side 3 -order 4
set_pad_physical_constraints -pad_name "vdd2right" -side 3 -order 5
set_pad_physical_constraints -pad_name "vdd1right" -side 3 -order 6
set_pad_physical_constraints -pad_name "vss1right" -side 3 -order 7
set_pad_physical_constraints -pad_name "vss2right" -side 3 -order 8
set_pad_physical_constraints -pad_name "pad_b_4" -side 3 -order 9
set_pad_physical_constraints -pad_name "pad_b_5" -side 3 -order 10
set_pad_physical_constraints -pad_name "pad_b_6" -side 3 -order 11
set_pad_physical_constraints -pad_name "pad_b_7" -side 3 -order 12
set_pad_physical_constraints -pad_name "pad_s_b" -side 3 -order 13

```

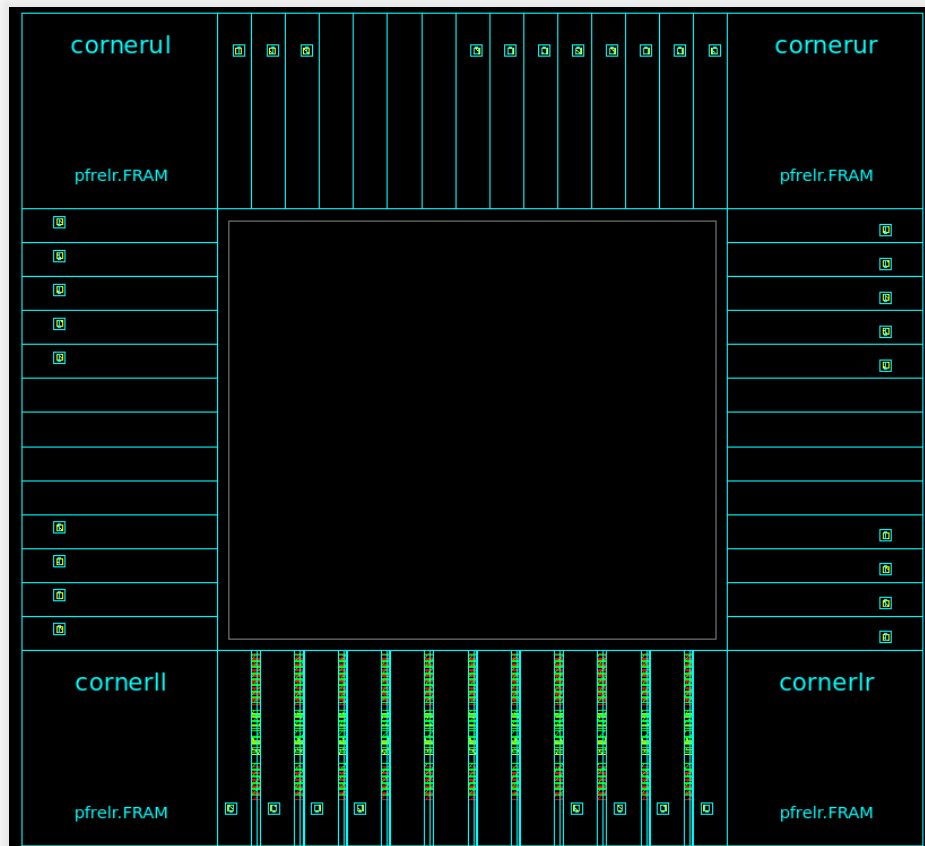
```

# Bottom side
set_pad_physical_constraints -pad_name "pad_res_0" -side 4 -order 1
set_pad_physical_constraints -pad_name "pad_res_1" -side 4 -order 2
set_pad_physical_constraints -pad_name "pad_res_2" -side 4 -order 3
set_pad_physical_constraints -pad_name "pad_res_3" -side 4 -order 4
set_pad_physical_constraints -pad_name "vdd2bottom" -side 4 -order 5
set_pad_physical_constraints -pad_name "vdd1bottom" -side 4 -order 6
set_pad_physical_constraints -pad_name "vss1bottom" -side 4 -order 7
set_pad_physical_constraints -pad_name "vss2bottom" -side 4 -order 8
set_pad_physical_constraints -pad_name "pad_res_4" -side 4 -order 9
set_pad_physical_constraints -pad_name "pad_res_5" -side 4 -order 10
set_pad_physical_constraints -pad_name "pad_res_6" -side 4 -order 11
set_pad_physical_constraints -pad_name "pad_res_7" -side 4 -order 12

```

4.2.2 Filler 填充以及 pg connect

由于底部 pad 数量最少，需要填充 filler，并进行 power ground connect



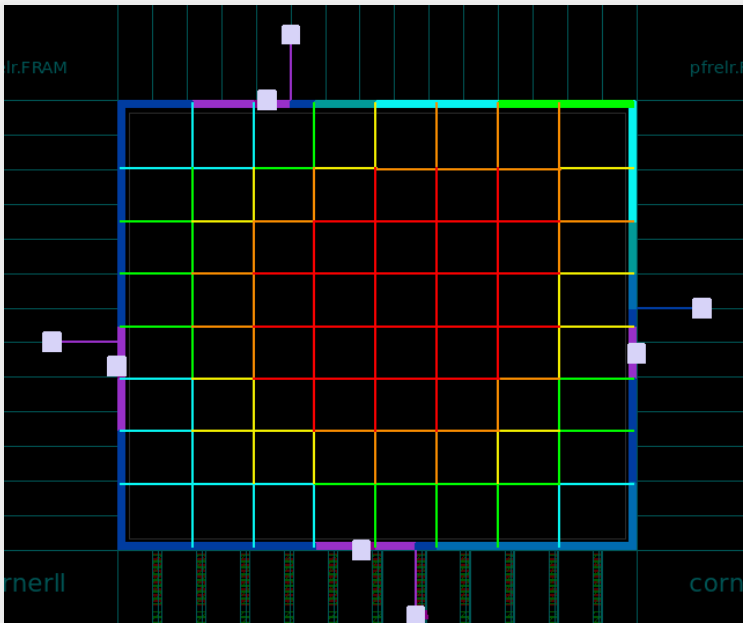
4.2.3 Congestion check

对版图进行 congestion 检查，可以看到没有 overflow 拥塞

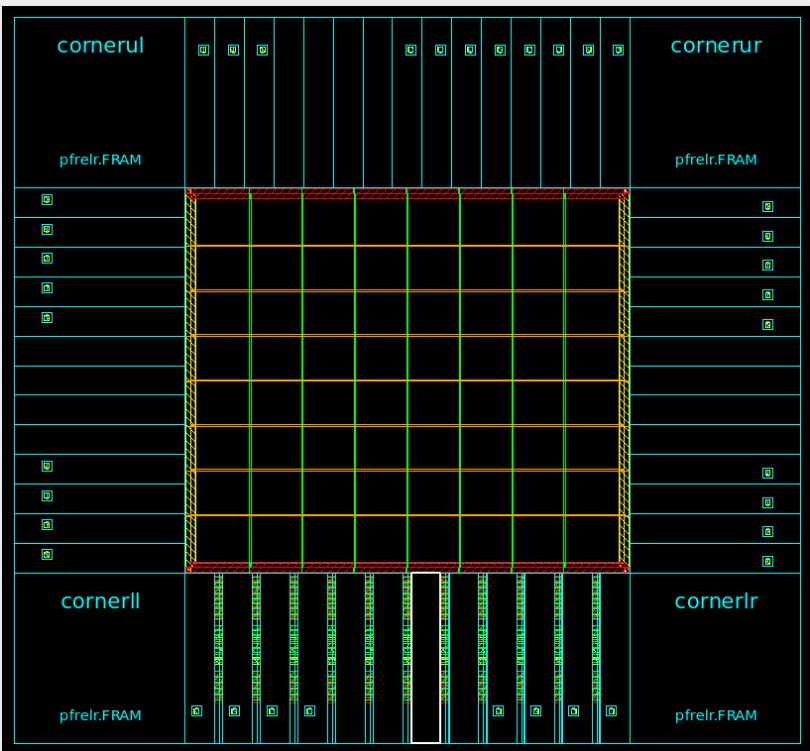
```
+++++
Overall Congestion Map Data:
+++++
There are 171570 GRCs (Global Route Cell) in this design
*****
***** GRC based congestion report *****
*****
0 GRCs with overflow: 0 with H overflow and 0 with V overflow
*****
+++++ Top 10 congested GRCs ++++++
+++++ Top 10 horizontally congested GRCs ++++++
+++++ Top 10 vertically congested GRCs ++++++
1
icc_shell>
```

4.2.4 PNS

进行 power network synthesize:



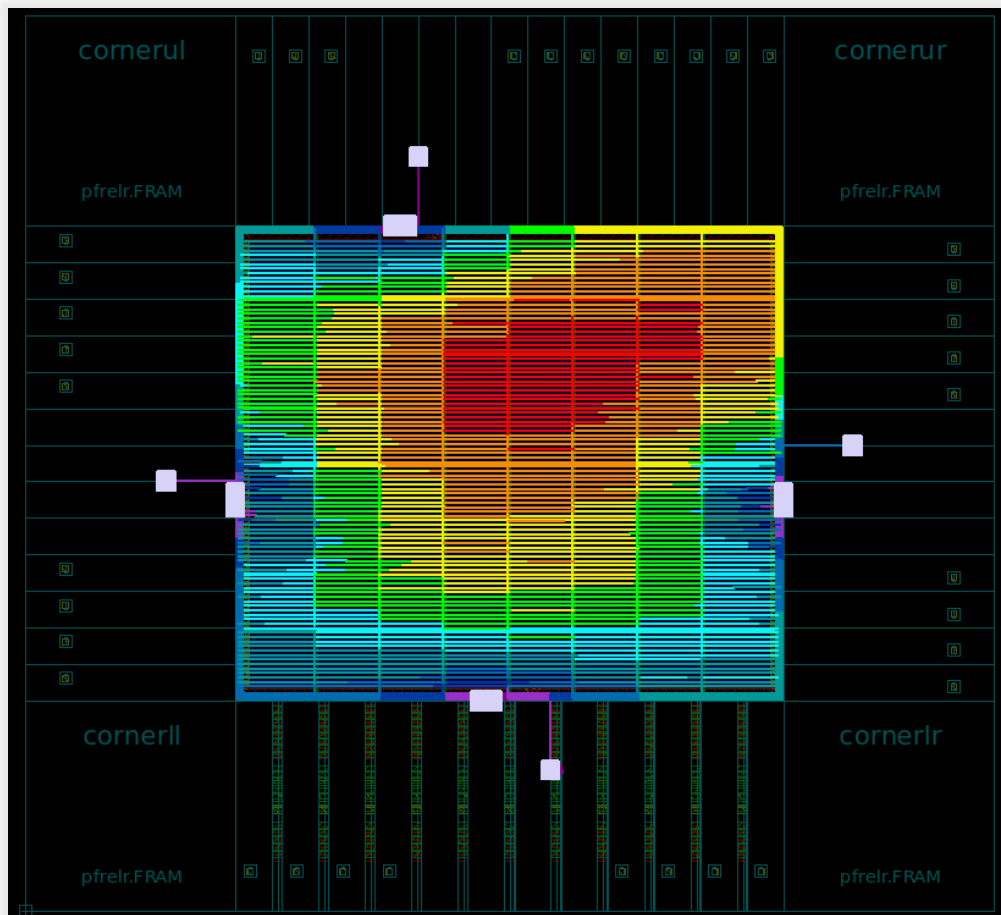
4.2.5 Power rail commit



4.2.6 IR drop 分析

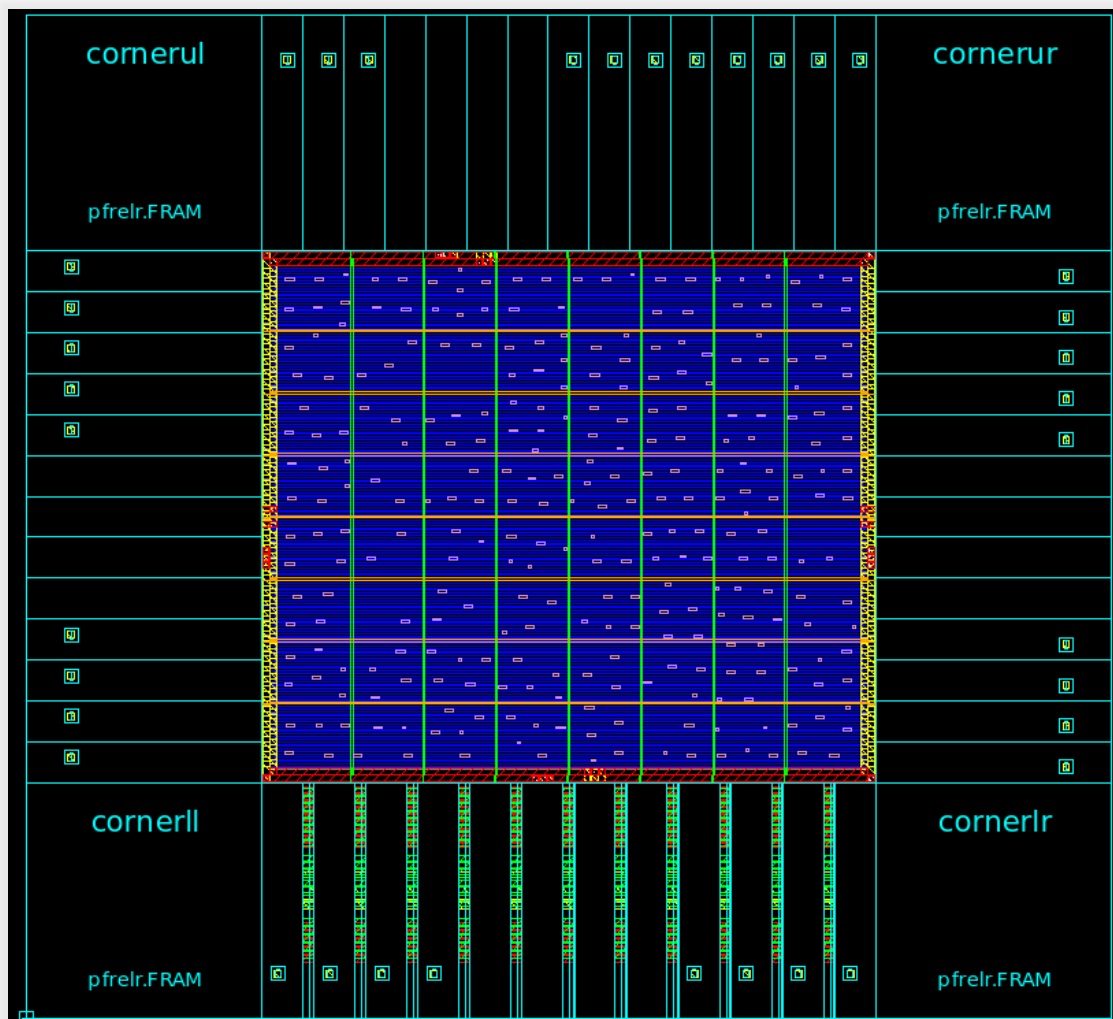
根据 IR drop 报告，没有超过压降阈值的路径：

```
1 *****
2 * Voltage Drop Violation Report
3 *
4 * Units:
5 *   Length Unit      : um
6 *   Length Scale     : 0.001
7 *   Voltage Unit     : mV
8 *
9 * Cell Name : F_PE_data_setup
10 * PG Net    : VDD
11 *
12 * Threshold: 65.982
13 *
14 * Format: [left bottom right top] on Layer is voltage
15 *
```



4.3 布局

4.3.1 标准单元布局



由于设计体量小，标准单元少而端口 pad 相对太多，导致芯片面积太大，而布局后标准单元分布十分稀疏。

在后期改进设计中，可以使用脉冲阵列等扩展设计，而对于有此带来的端口太多的问题则可以采用地址选择的方法，对 io 端口进行复用，从而减少端口 pad 的使用，进而减少芯片的面积。

4.3.2 布局结果报告

(1) 面积报告

Combinational area:	127586.338135
Buf/Inv area:	126816.088135
Noncombinational area:	1774.750000
Macro/Black Box area:	0.000000
Net Interconnect area:	774.495626
Total cell area:	129361.088135
Total area:	130135.583761

Hierarchical area distribution

Hierarchical cell	Global cell area		Local cell area			Design
	Absolute Total	Percent Total	Combi-national	Noncombi-national	Black-boxes	
F_PE	129361.0881	100.0	127312.3381	1189.7500	0.0000	F_PE
bb0	74.0000	0.1	29.0000	45.0000	0.0000	BitBrick_15
bb1	55.0000	0.0	17.5000	37.5000	0.0000	BitBrick_14
bb10	38.5000	0.0	8.5000	30.0000	0.0000	BitBrick_5
bb11	56.5000	0.0	19.0000	37.5000	0.0000	BitBrick_4
bb12	55.0000	0.0	17.5000	37.5000	0.0000	BitBrick_3
bb13	55.7500	0.0	18.2500	37.5000	0.0000	BitBrick_2

(2) 功耗报告

Global Operating Voltage = 1.08					
Power-specific unit information :					
Voltage Units = 1V					
Capacitance Units = 1.000000pf					
Time Units = 1ns					
Dynamic Power Units = 1mW (derived from V,C,T units)					
Leakage Power Units = 1pW					

Hierarchy	Switch Power	Int Power	Leak Power	Total Power	%
F_PE	0.184	164.236	1.72e+07	164.437	100.0
bb8 (BitBrick_7)	1.01e-03	5.28e-03	2.24e+05	6.52e-03	0.0
bb9 (BitBrick_6)	9.73e-04	5.12e-03	2.24e+05	6.32e-03	0.0
bb10 (BitBrick_5)	1.09e-03	5.32e-03	2.24e+05	6.64e-03	0.0
bb11 (BitBrick_4)	2.14e-03	7.59e-03	3.23e+05	1.00e-02	0.0
bb12 (BitBrick_3)	1.97e-03	6.79e-03	3.21e+05	9.08e-03	0.0
bb13 (BitBrick_2)	2.18e-03	6.78e-03	3.24e+05	9.28e-03	0.0
bb14 (BitBrick_1)	2.32e-03	7.05e-03	3.27e+05	9.69e-03	0.0
bb15 (BitBrick_0)	1.91e-03	8.86e-03	4.37e+05	1.12e-02	0.0
bb0 (BitBrick_15)	1.35e-03	7.78e-03	4.36e+05	9.57e-03	0.0
bb1 (BitBrick_14)	1.55e-03	6.51e-03	3.16e+05	8.38e-03	0.0

(3) 时序报告

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	0.40	0.40 r
bitwidth[1] (in)	0.00	0.40 r
pad bw 1/CIN (pc3d01)	0.97 @	1.37 r
U346/ZN (nr02d0)	0.17 @	1.55 f
U360/ZN (nd02d0)	0.11 &	1.66 r
U254/ZN (nd02d1)	0.08 &	1.75 f
U256/ZN (inv0d2)	0.04 &	1.78 r
U255/ZN (nd02d2)	0.08 &	1.86 f
U204/ZN (invbd7)	0.05 &	1.91 r
U205/ZN (invbdf)	0.07 &	1.98 f
pad res 3/PAD (pc3o05)	1.90 &	3.88 f
result[3] (out)	0.00	3.88 f
data arrival time		3.88
clock clk (rise edge)	2.00	2.00
clock network delay (ideal)	0.00	2.00
clock reconvergence pessimism	0.00	2.00
clock uncertainty	-0.10	1.90
output external delay	-0.40	1.50
data required time		1.50
data required time		1.50
data arrival time		-3.88
slack (VIOLATED)		-2.38

观察到 slack 为负数，违反了时序；而且存在的主要延时为 pad 带来的组合逻辑延时。在 pad 与芯片标准内部之间添加一组寄存器后，时序报告如下：

```

50
57 Startpoint: result_reg_3
58      (rising edge-triggered flip-flop clocked by clk)
59 Endpoint: result[3] (output port clocked by clk)
70 Path Group: OUTPUTS
71 Path Type: max
72
73 Point                                Incr      Path
74 -----
75 clock clk (rise edge)                0.00      0.00
76 clock network delay (ideal)          0.00      0.00
77 result_reg_3/CP (sdnrq1)             0.00      0.00 r
78 result_reg_3/Q (sdnrq1)             0.81      0.81 f
79 pad_res_3/PAD (pc3o05)              2.11      2.92 f
80 result[3] (out)                      0.00      2.92 f
81 data arrival time                    delay      2.92
82
83 clock clk (rise edge)                2.00      2.00
84 clock network delay (ideal)          0.00      2.00
85 clock uncertainty                    -0.14     1.86
86 output external delay                -0.20     1.66
87 data required time                   1.66
88 -----
89 data required time                   1.66
90 data arrival time                    -2.92
91 -----
92 slack (VIOLATED)                     -1.26
93

```

pcq delay

delay

关键路径的产生仍然源于 pad。注意到寄存器输出结果到 pad 输入存在 propagation to clock delay 为 0.81ns，而 pad 本身的组合逻辑延时为 2.11ns，此时只能通过降频的方法来解决时序问题：

根据： $T0 \geq pcq\ delay + pad\ delay + uncertainty\ delay + output\ delay$ 计算得 $T0 \approx 3.3ns$ ，即时钟频率为 300M。

修改约束脚本，降频率降低到 300M 之后，成功通过时序验证

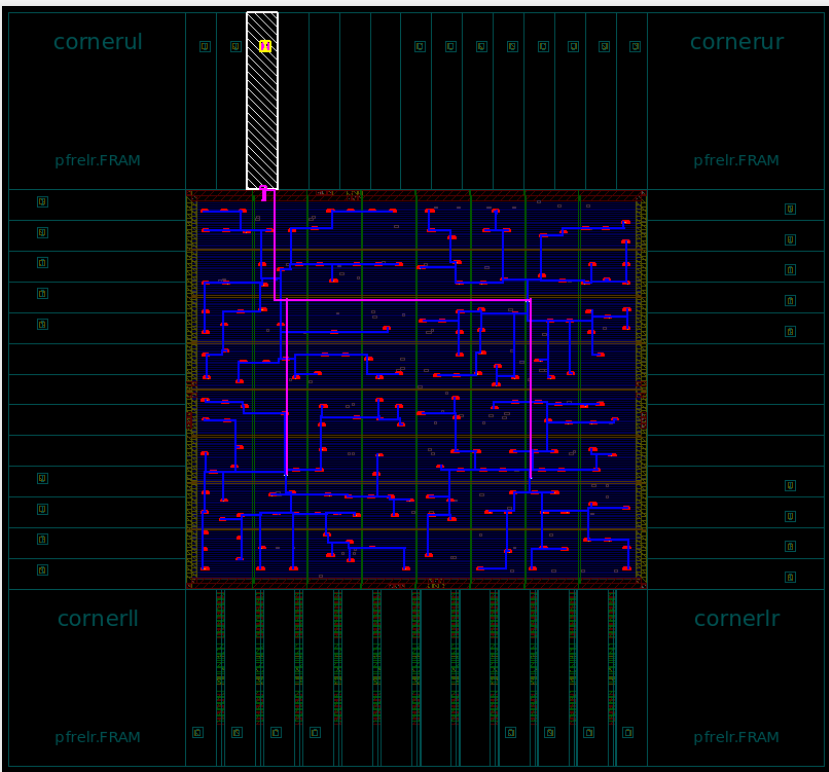
```

1
2 *****
3 Report : timing
4     -path full
5     -delay max
6     -slack_lesser_than 0.00
7     -max_paths 10
8 Design : F_PE
9 Version: L-2016.03-SP1
0 Date   : Sun Jun 22 00:26:18 2025
1 *****
2
3 Operating Conditions: cb13fs120_tsmc_max Library: cb13fs120_tsmc_max
4 No paths.
5

```

4.4 时钟树综合

4.4.1 CTS



4.4.2 时序报告

1	Startpoint: result_reg_8		
2	(rising edge-triggered flip-flop clocked by clk)		
3	Endpoint: result[8] (output port clocked by clk)		
4	Path Group: OUTPUTS		
5	Path Type: max		
6			
7	Point	Incr	Path
8	-----	-----	-----
9	clock clk (rise edge)	0.00	0.00
10	clock network delay (propagated)	1.22	1.22
11	result_reg_8/CP (sdnrq2)	0.00	1.22 r
12	result_reg_8/Q (sdnrq2)	0.38	1.60 f
13	U149/ZN (invbdf)	0.05 *	1.65 r
14	U150/ZN (invbdf)	0.07 *	1.72 f
15	pad_res_8/PAD (pc3o05)	1.90 *	3.62 f
16	result[8] (out)	0.00 *	3.62 f
17	data arrival time		3.62
18			
19	clock clk (rise edge)	3.33	3.33
20	clock network delay (ideal)	0.00	3.33
21	clock uncertainty	-0.10	3.23
22	output external delay	-0.20	3.03
23	data required time		3.03
24	-----	-----	-----
25	data required time		3.03
26	data arrival time		-3.62
27			
28	slack (VIOLATED)		-0.59
29			

在时钟树综合之后，得到时钟网络延时，在关键路径中延时为 1.22ns，但由于在输出端口时钟延时是理想时钟延时，即为 0，导致出现了时序违例的问题。即使经过时钟布局优化之后 slack 仍然为负。此时只能继续增加 0.5ns 的时钟周期，最终显示通过。此时时钟频率为 263M。由于工艺库的限制，在考虑了时钟网络延时之后，时钟周期最低也只能为 $T = T_0 - tc$, $T_0 \approx 3.3ns$ 左右，即可运行的最高频率小于 300M。本设计已实现至工艺库的极限

4.5 布线

4.5.1 布线实现

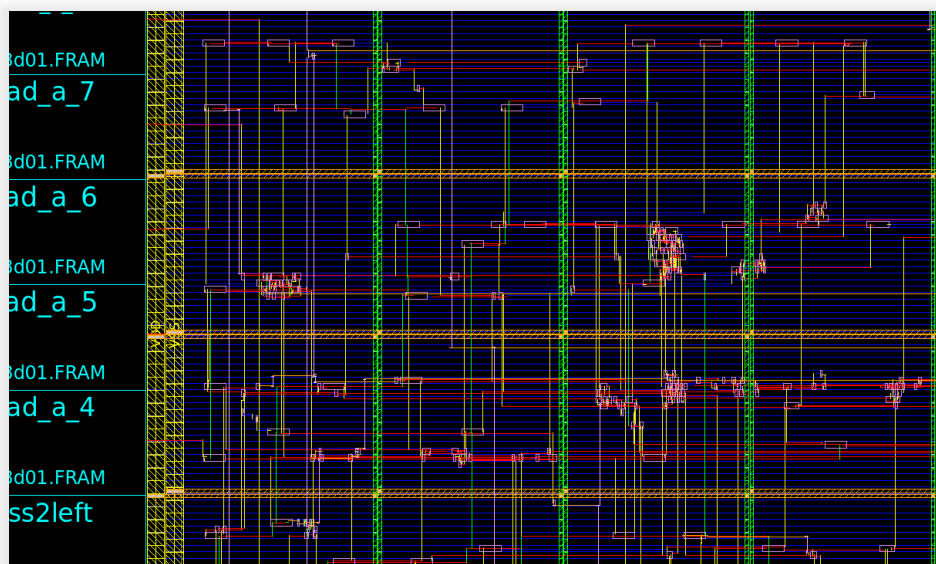


图 1：布线局部图

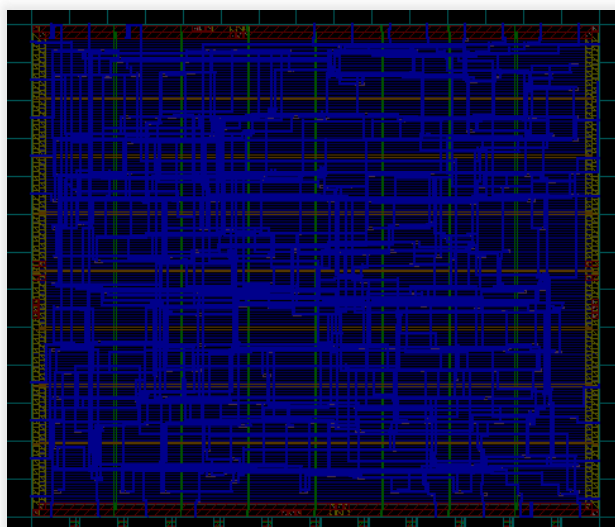


图 2：net capacitance

4.5.2 时序分析

时序报告显示没有违例，全部满足时序约束

4.6 DRC 验证

```
Verify Summary:

Total number of nets = 963, of which 0 are not extracted
Total number of open nets = 0, of which 0 are frozen
Total number of excluded ports = 0 ports of 0 unplaced cells connected to 0 nets
                                0 ports without pins of 0 cells connected to 0 nets
                                0 ports of 0 cover cells connected to 0 non-pg nets
Total number of DRCs = 0
Total number of antenna violations = no antenna rules defined
Total number of voltage-area violations = no voltage-areas defined
Total number of tie to rail violations = not checked
Total number of tie to rail directly violations = not checked
```

DRC 数量为 0，证明 DRC 验证全部通过

4.7 LVS 验证

```
ERROR : OUTPUT PortInst R_180 Q doesn't connect to any net.

ERROR : OUTPUT PortInst R_147 QN doesn't connect to any net.

ERROR : OUTPUT PortInst R_150 QN doesn't connect to any net.

ERROR : OUTPUT PortInst R_157 QN doesn't connect to any net.

ERROR : OUTPUT PortInst R_176 QN doesn't connect to any net.

ERROR : OUTPUT PortInst R_162 Q doesn't connect to any net.

ERROR : There are more errors than default Max Error Number 20.
** Total Floating ports are 20.
** Total Floating Nets are 0.
** Total SHORT Nets are 0.
** Total OPEN Nets are 0.
** Total Electrical Equivalent Error are 0.
** Total Must Joint Error are 0.

-- LVS END : --
Elapsed =    0:00:00, CPU =    0:00:00
```

LVS 验证显示存在 20 个未连接网线的输出端口

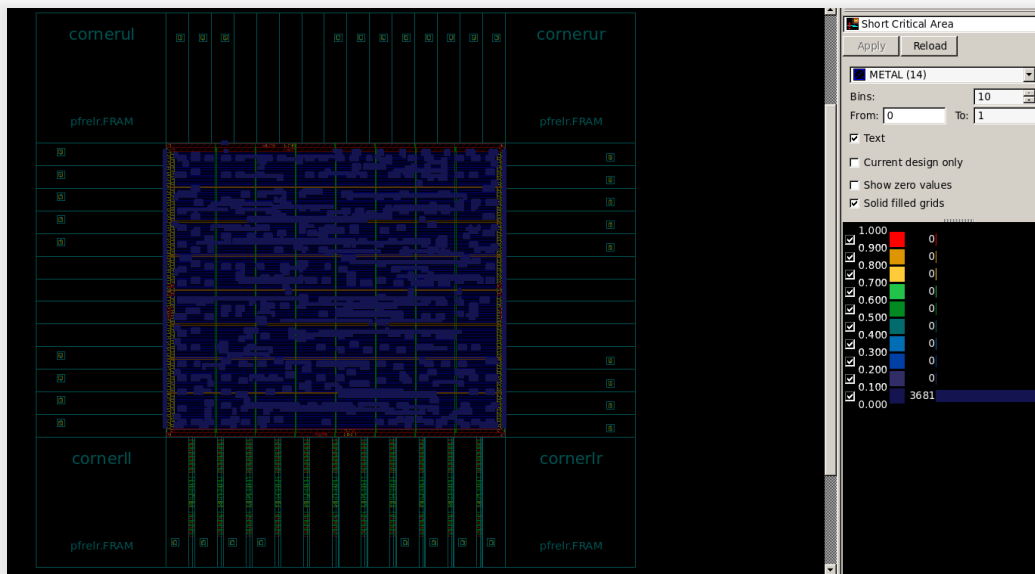
```

1172 ad01d0 0182 ( .A(n122), .B(n867), .CI(n597), .CO(n482), .S(n480) );
1173 sdnrb1 R_179 ( .D(n476), .SD(1'b0), .SC(1'b0), .CP(net_clk), .Q(n151) );
1174 sdnrb1 R_180 ( .D(n473), .SD(1'b0), .SC(1'b0), .CP(net_clk), .QN(n453) );
1175 sdnrb1 R_181 ( .D(n471), .SD(1'b0), .SC(1'b0), .CP(net_clk), .Q(n28) );
1176 ad01d1 U78 ( .A(n29), .B(n28), .CI(\DP_OP_57J1_123_753/n183), .CO(n215),
1177 S(n46) );

```

对比检查网表文件之后，确认这些浮空端口的输出都不需要使用，所以 LVS 验证也全部通过

查看关键区域：



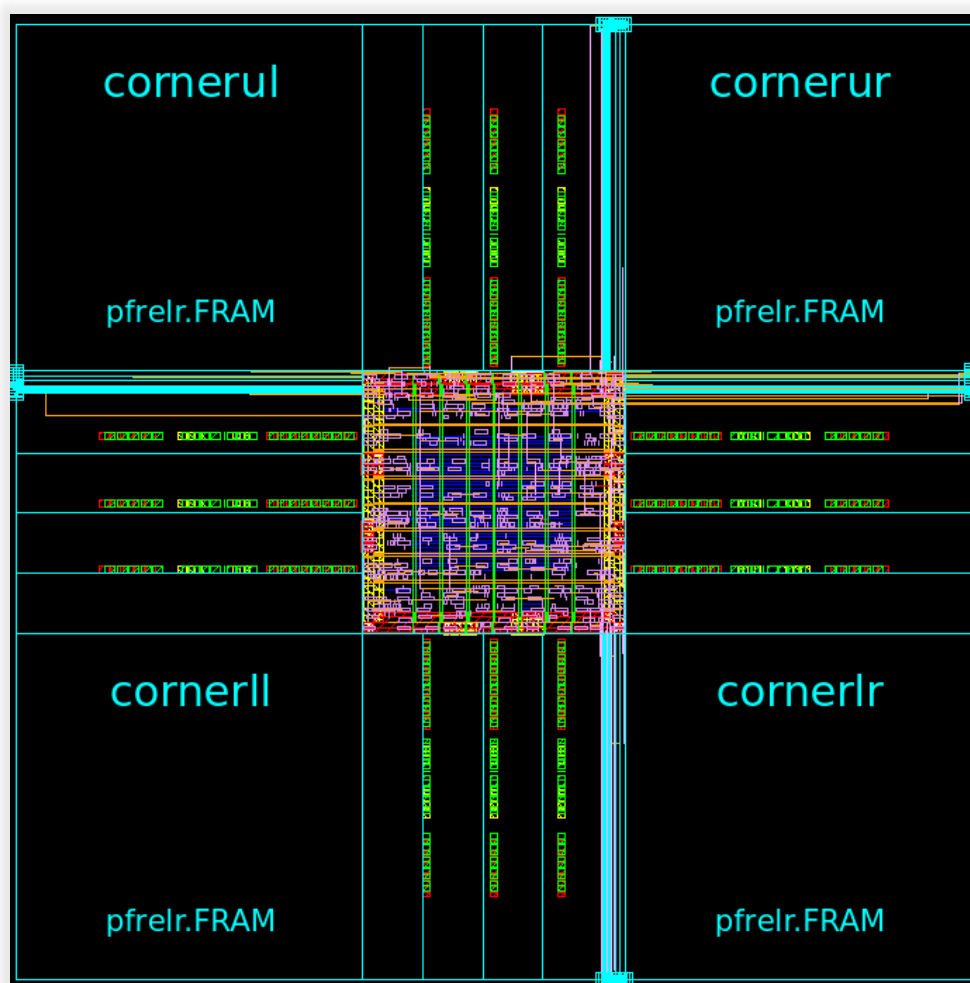
由于设计很小，关键区域几乎没有

4.8 版图文件导出

在减少关键区域和插入金属填充物后，再通过 DRC 和 LVS 检查，最后导出 GDSII 版图数据文件

第5部分 实验过程中遇到的问题

- (1) 为了增加时钟主频，我们将组合逻辑电路转换为了 3 级流水线的时序逻辑电路。在这个过程中，我们遇到了数据时序上的错误。最后我们经过一级一级仿真，逐步排查，最终解决了问题。
- (2) 在后端综合时，时序约束的设置不合理导致了后期的时序上的错误，需要切实地根据工艺库编写时序约束脚本。
- (3) 在后端布局规划的时候，所使用的 DC 综合出来的网表文件并没有 IO PAD，即产生了如图所示的问题：



IO 端口出现了重叠。为了解决这个问题，需要修改综合出来的网表文件，即添加对 IO PAD 的例化。


```

38 module F_PE ( clk, a, b, s_a, s_b, bitwidth, result );
39     input [7:0] a;
40     input [7:0] b;
41     input [1:0] bitwidth;
42     output [15:0] result;
43     input clk, s_a, s_b;
44
45
46     wire net_clk;
47     wire [7:0] net_a;
48     wire [7:0] net_b;
49     wire [1:0] net_bitwidth;
50     wire [15:0] net_result;
51     wire net_s_a, net_s_b;
52     wire [7:0] a_to_r;
53     wire [7:0] b_to_r;
54     wire [1:0] bitwidth_to_r;
55     wire [15:0] result_from_r;
56     wire s_a_to_r, s_b_to_r;
57
58     pc3d01 pad_clk ( .PAD(clk), .CIN(net_clk) );
59
60     pc3d01 pad_a_0 ( .PAD(a[0]), .CIN(a_to_r[0]) );
61     pc3d01 pad_a_1 ( .PAD(a[1]), .CIN(a_to_r[1]) );
62     pc3d01 pad_a_2 ( .PAD(a[2]), .CIN(a_to_r[2]) );
63     pc3d01 pad_a_3 ( .PAD(a[3]), .CIN(a_to_r[3]) );
64     pc3d01 pad_a_4 ( .PAD(a[4]), .CIN(a_to_r[4]) );
65     pc3d01 pad_a_5 ( .PAD(a[5]), .CIN(a_to_r[5]) );
66     pc3d01 pad_a_6 ( .PAD(a[6]), .CIN(a_to_r[6]) );

```

但由于 PAD 模块所带来的严重的时延，可能会导致时序上的违例，此时需要在 PAD 以及芯片内部之间添加寄存器来减少时序延迟。在设计上相当于实现了 5 级的流水线电路设计。

第6部分 总结

本次 EDA 课设我们实现了 PE 单元的从前端到测试仿真再到后端综合以及版图设计的整个 IC 设计流程。通过对 PE 单元乘法器的学习与设计，我们学会了 BIT-FUSION 算法，并且初步接触到了什么脉动阵列及其具体作用。在对后端的实现中，我们学以致用，将实验课上对 DC 以及 ICC 的学习进行了实践，成功实现了后端的设计。经过这次实践，我们明白了数字 IC 的整个设计流程，并且深刻认识到时序和时钟对于数字电路设计的重要性，极大地提高了我们数字电路设计的能力。

非常感谢老师的细心教导以及帮助！

第7部分 引用文献

- [1] Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Networks
- [2] DC_guide_2013.03
- [3] ICC_guide_2010.12